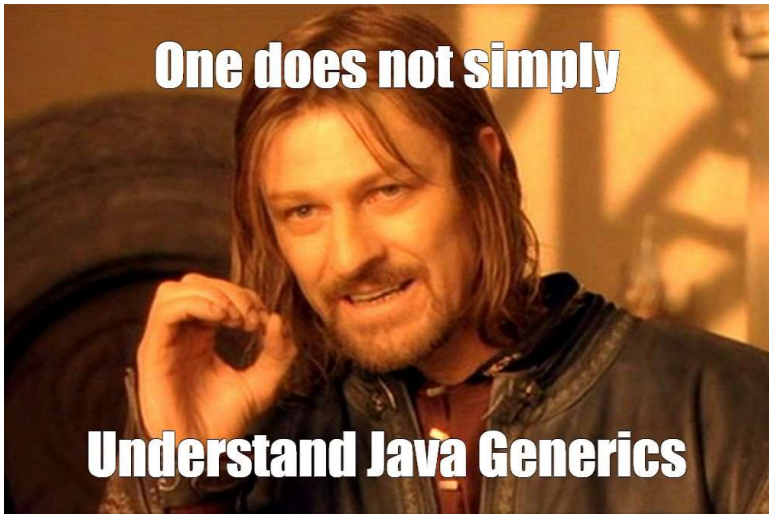
Modul 5

Generic Class



Praktikum Pemrograman Berorientasi Objek

Semester Genap T.A. 2025 / 2026

Program Studi Informatika

Universitas Atma Jaya Yogyakarta

Tujuan

Praktikan dapat memahami konsep Generic Class serta dapat mengimplementasikannya dalam Bahasa Pemrograman Java.

Penjelasan

Generic Class adalah konsep dalam pemrograman yang memungkinkan sebuah kelas memiliki tipe data yang dapat disesuaikan. Dengan menggunakan generic, satu kelas dapat digunakan untuk berbagai jenis tipe data tanpa perlu membuat kelas baru untuk setiap tipe.

Hal ini membuat kode menjadi lebih rapi, efisien, dan menghindari pengulangan penulisan kode yang sama.

Berikut contoh kode dari Generic Class:

```java
public class GenericClass <T> {

    private T value;

    public GenericClass(T value) {
        this.value = value;
    }

    public T getValue(){
        return value;
    }

    public void setValue(T value){
        this.value = value;
    }
}
```

***T** adalah parameter formal yang dapat digantikan nantinya saat pendefinisian objek*

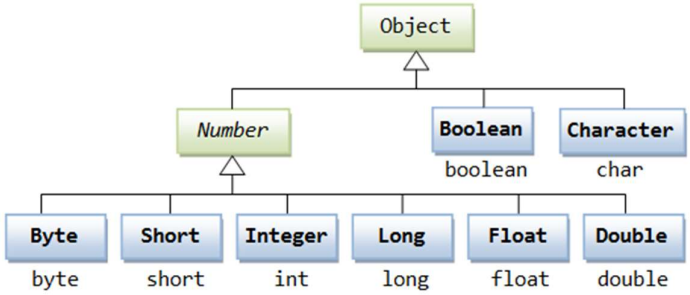
Parameter formal pada Generic **Class tidak harus menggunakan huruf T**. Programmer bebas menggunakan nama apa saja sesuai aturan penamaan variabel. Namun, untuk mengikuti **konvensi umum** dan mempermudah pembacaan kode, biasanya digunakan satu huruf kapital seperti T, E, K, atau V.

Parameter tersebut harus didefinisikan **di dalam tanda kurung sudut < > setelah nama class**. Nama parameter inilah yang digunakan sebagai pengganti tipe data di dalam class agar bersifat fleksibel.

Perlu diketahui bahwa Generic hanya berlaku untuk tipe **data referensi**. Namun, di dalam Generic Class tetap diperbolehkan menggunakan **tipe data primitif seperti int atau float** untuk kebutuhan lain dalam program.Contoh penerapan generic class pada main class:

```java
public static void main(String[] args) {
    GenericClass<Integer> genericInt = new GenericClass<>(291);
    System.out.println(genericInt.getValue); // 291

    GenericClass<String> genericStr = new GenericClass<>("Sinay");
    System.out.println(genericStr.getValue); // Sinay

  /*
    Bisa juga menggunakan cara
    GenericClass<String> genericStr = new GenericClass<String>("Sinay");
    namun, sejak Java versi 7 ke atas, penulisan yang disarankan adalah
    mengosongkan kurung siku atau diamond kosong <> tersebut
  */
}
```

Perlu diperhatikan pada contoh penerapan di atas, dalam pembuatan objek dari generic class, tipe data aktual diletakkan di dalam diamond operator (< >). Tetapi perlu diketahui juga bahwa generic class wajib menggunakan **parameter tipe reference (dari kelas buatan sendiri maupun bawaan Java), atau *object wrapper* (digunakan untuk membungkus tipe data primitive menjadi bentuk objek)**, seperti pada gambar berikut:



Karena String bukan merupakan primitive, sehingga tidak diletakan pada diagram di atas. String sudah merupakan representasi objek di Java sehingga dapat langsung digunakan seperti gambar di main class.

Generic class juga dapat dimodifikasi sesuai kebutuhan sang programmer untuk menetapkan banyaknya parameter formal yang diperlukan tergantung keperluan programmer, sehingga apabila memerlukan parameter lebih dari satu dapat didefinisikan sebagai berikut:

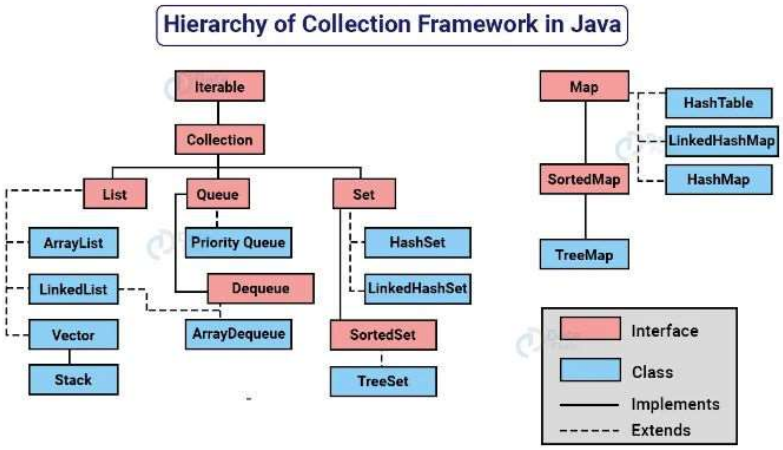
```java
public class TwoPair <K, V> {
    private K valueK;
    private V valueV;

    public TwoPair(K valueK, V valueV) {
        this.valueK = valueK;
        this.valueV = valueV;
    }

    public K getValueK() {
        return valueK;
    }

    public V getValueV() {
        return valueV;
    }

    public void setValueK(K valueK) {
        this.valueK = valueK;
    }

    public void setValueV(V valueV) {
        this.valueV = valueV;
    }
}
```

Contoh penerapan pada main class:

```java
public static void main(String[] args) {
    TwoPair<Integer, Integer> twoInt = new TwoPair<>(9, 1);
    System.out.println(twoInt.getValueK()); // 9
    System.out.println(twoInt.getValueV()); // 1

    TwoPair<String, Integer> diffType = new TwoPair<>("Sinay", 1);
    System.out.println(diffType.getValueK()); // Sinay
    System.out.println(diffType.getValueV()); // 1

    TwoPair<TwoPair<String, Integer>, Double> inside =
      new TwoPair<>(difftype,
                    9.00);
    System.out.println(inside.getValueK().getValueK()); // Sinay
    System.out.println(inside.getValueK().getValueV()); // 1
    System.out.println(inside.getValueV()); // 9.00

}
```

Pada contoh penerapan TwoPair, kita dapat mendefinisikan tipe data aktual seperti yang sudah dijelaskan sebelumnya, dan dapat langsung menggunakan generic class sesuai dengan tipe data yang sudah didefinisikan. Kita juga dapat menggunakan generic class sebagai tipe dari generic class lainnya yang dapat dilihat pada contoh di atas. Sehingga penggunaan konsep ini sangat fleksible tergantung keperluan dari sang programmer.

Java Collections



Collection merupakan framework atau kerangka kerja yang merepresentasikan suatu struktur data yang sudah ada sebelumnya. Collection bertujuan untuk mempermudah sang programmer untuk menerapkan struktur data tertentu tanpa perlu mengimplementasikannya sendiri, sehingga dapat dibilang suatu solusi praktis dan efisien untuk suatu masalah yang berulang.

Loh? Kok membahas struktur data?

Karena, collection menggunakan konsep generic class dalam penggunaannya, sehingga sekalian membahas ini, dan mungkin dapat membantu pengerjaan UGD maupun Tugas Besar kedepannya.

Collection yang akan dibahas pada modul ini adalah ArrayList, yang pada dasarnya adalah linked list yang bersifat seperti array, jadi fleksible dalam sizenya namun dapat diakses menggunakan indeks seperti array. Berikut contoh penggunaannya:

```
import java.util.ArrayList;
import java.util.List;
```

Lakukan import terlebih dahulu ArrayList dan List untuk dapat digunakan pada class yang diinginkan, contoh import dan pembuatan variable ArrayList:

```
import java.util.ArrayList;
import java.util.List;

public class Main {
  public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();
        // Contoh pembuatan list integer
  }
}
```

Terdapat method yang disediakan untuk ArrayList, sebagai berikut :

Method	Tipe Balikan	Deskripsi
`.add(<Type> data)`	void	Menambahkan data ke dalam list.
`.get(int index)`	int	Membaca data pada index tertentu.
`.contains(<Type> data)`	boolean	Mengecek apakah data terdapat pada list.
`.remove(int index)`	void	Menghapus data pada index tertentu.
`.set(int index, <Type> data)`	void	Mengubah data pada index tertentu dengan data tertentu.
`.indexOf(<Type> dataToFind)`	int	Mendapatkan index dari data yang ingin dicari.

Guided Generic Class

1. Buatlah projek pada NetBeans lalu beri nama dengan **GD5_X_5digitNPM**

   X = Huruf Kelas

2. Lalu buatlah **empat** kelas baru di package yang sama dengan main, sebagai berikut:

```java
public class GenericClass <T> {

    private T value;

    public GenericClass(T value) {
        this.value = value;
    }

    public T getValue(){
        return value;
    }

    public void setValue(T value){
        this.value = value;
    }
}
```

GenericClass.java

```java
public class TwoPair <K, V> {
    private K valueK;
    private V valueV;

    public TwoPair(K valueK, V valueV) {
        this.valueK = valueK;
        this.valueV = valueV;
    }

    public K getValueK() {
        return valueK;
    }

    public V getValueV() {
        return valueV;
    }

    public void setValueK(K valueK) {
        this.valueK = valueK;
    }

    public void setValueV(V valueV) {
        this.valueV = valueV;
    }
}
```

TwoPair.java

```java
public class Mahasigma {
    private String nama;
    private String npm;
    private double ipk;

    public Mahasigma(String nama, String npm, double ipk) {
        this.nama = nama;
        this.npm = npm;
        this.ipk = ipk;
    }
    // untuk guided kali ini tidak memerlukan getter setter kalian bisa menambahkan jika kalian
       memerlukannya
    @Override
    public String toString(){
        return "Nama : " +this.nama+"\n"
                + "NPM : "+this.npm+"\n"
                + "IPK : "+this.ipk+"\n";
    }
    /*
        Kita menggunakan method bawaan dari sebuah class, yaitu toString(), untuk menampilkan seluruh
        isi objek.
        Method ini bisa menggantikan method seperti showData() yang biasanya dibuat sendiri.

        Di dalam main, kita bisa langsung mencetak objek menggunakan System.out.println(obj);
        tanpa perlu menuliskan .toString() secara langsung, karena Java akan otomatis memanggil method
        tersebut.

        Method toString() perlu diberi anotasi @Override karena kita ingin mengganti perilaku default
        dari class Object.
        Jika tidak dioverride, yang ditampilkan hanyalah alamat memori objek, bukan isi datanya.
    */

}
```

Mahasigma.java

```java
public class GenericList <T> {
    private List<T> list;

    public GenericList(){
        list = new ArrayList<>();
    }

    public void add(T data){
        list.add(data);
    }

    public void remove(int index){
        list.remove(index);
    }

    public int getIndex(T data){
        return list.indexOf(data);
    }

    public void update(int index, T data){
        list.set(index, data);
    }

    public boolean isFound(T data){
        return list.contains(data);
    }

    public void print(){
        /*
            ini adalah range based loop
            loop ini akan melakukan print sesuai jumlah data list
        */
        for(T data : list){
            /*
                jika kalian bingung kenapa langsung data pada printnya
                pada java dia akan langsung memanggil method toString secara otomatis
                , jika kalian ingin tetap memanggil dengan cara data.toString() masalah
            */
            System.out.print(data);
            //System.out.print(data.toString()); ini juga tidak masalah
        }
        // Cara lain menggunakan interactive loop
        /*
            for(int i=0; i<list.size(); i++){
            System.out.println(list.get(i));
            System.out.println(list.get(i).toString()); ini juga boleh
        }
        */
        System.out.println();
    }
}
```

GenericList.java

Note :

- Object Wrapper sudah memiliki method toString() yang sesuai dengan tipe data yang dibuat, apabila Integer otomatis akan menampilkan Integer, dsb.

```java
public static void main(String[] args) {
        GenericClass<String> genericStr = new GenericClass<String>("nama praktikan"); //diubah
        System.out.println(genericStr.getValue()); //nama praktikan

        GenericClass<Double> genericDbl = new GenericClass<Double>(3.14);
        System.out.println(genericDbl.getValue()); //3.14

        TwoPair<Integer, Integer> twoInt = new TwoPair<Integer, Integer>(1, 2);
        System.out.println(twoInt.getValueK()); // 1
        System.out.println(twoInt.getValueV()); // 2

        TwoPair<Integer, String> diffType = new TwoPair<>(1, "sinay");
        System.out.println(diffType.getValueK()); // 1
        System.out.println(diffType.getValueV()); // sinay

        TwoPair<TwoPair<Integer, String>, Double> inside = new TwoPair<>(diffType, 4.00);
        System.out.println(inside.getValueK().getValueK()); // 1
        System.out.println(inside.getValueK().getValueV()); // sinay
        System.out.println(inside.getValueV()); // 4.00

        GenericList<Integer> npm = new GenericList<>();
        // masukan 5 digit npm
        npm.add(1);
        npm.add(2);
        npm.add(2);
        npm.add(9);
        npm.add(1);
        npm.print(); // 12291

        GenericList<Character> atma = new GenericList<>();
        atma.add('A');
        atma.add('T');
        atma.add('M');
        atma.add('A');
        atma.print(); // ATMA

        Mahasigma sinay = new Mahasigma("sinay", "230712291", 4.0);
        Mahasigma alex = new Mahasigma("alexander wongso", "230712519", 4.0);
        Mahasigma anda = new Mahasigma("nama praktikan", "npm praktikan", 4.0);

        GenericList<Mahasigma> mhs = new GenericList<>();
        mhs.add(sinay);
        mhs.add(alex);
        mhs.add(anda);
        mhs.print();
    }

}
```

Main

3. Setelah selesai menulis ketiga class baru serta code pada main, dapat langsung kalian run dan hasilnya adalah sebagai berikut :

```
run:
nama praktikan
3.14
1
2
1
sinay
1
sinay
4.0
12291
ATMA
Nama : sinay
NPM : 230712291
IPK : 4.0
Nama : alexander wongso
NPM : 230712519
IPK : 4.0
Nama : nama praktikan
NPM : npm praktikan
IPK : 4.0
```

Ubah Nama Praktikan, dan NPM sesuai yang kalian miliki

Format Penamaan :

GD5_X_5digitNPM.zip

X = Huruf Kelas

Hint UGD / Tugas!

Silahkan untuk mencoba pemanfaatan modul Pewarisan, Polimorfisme, dan Relasi untuk pengerjaan Generic Class, serta penerapan ArrayList pada program.

Hint Bonus!

Bagi yang ingin tantangan, coba pelajari cara pembuatan struktur data pada Bahasa Pemrograman Java, struktur data tidak jauh – jauh dari yang ada di ISD tahun lalu.